

Padrões de Projeto (Composite)

QXD0068 - Reuso de Software

Universidade Federal do Ceará - *Campus* Quixadá

Prof. Francisco Victor da Silva Pinheiro
victorpinheiro@ufc.br



Agenda

- Introdução
- Padrão Composite
- Problema
- Solução
- Analogia do Mundo Real
- Estrutura do Padrão
- Exemplo em Kotlin e Python
- Aplicabilidade
- Prós e Contras
- Implementação



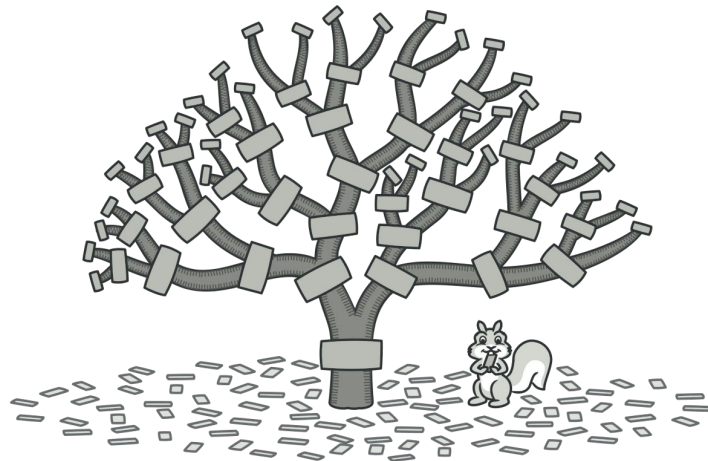
Composite

Introdução

- O padrão de projeto Composite pertence à categoria dos padrões estruturais e tem como objetivo tratar objetos individuais e grupos de objetos de forma uniforme.
- Ele permite organizar objetos em estruturas hierárquicas em forma de árvore, onde cada nó pode representar um elemento simples (folha) ou composto (contêiner).
- Esse padrão é especialmente útil quando queremos aplicar uma mesma operação a elementos simples e compostos, sem precisar distinguir suas classes concretas.

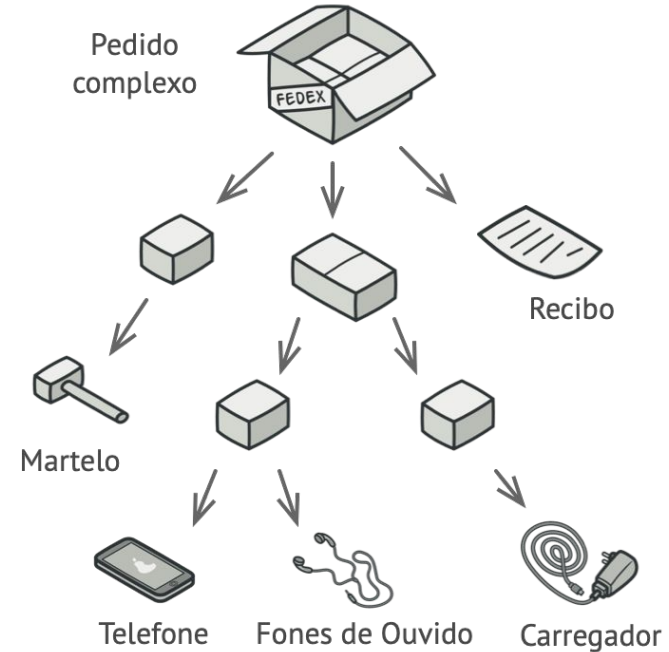
Padrão Composite

- O Composite define uma interface comum para todos os objetos da árvore, permitindo que o cliente interaja com elementos simples e compostos da mesma maneira.
- Assim, o sistema ganha flexibilidade para representar relações “parte-todo” — por exemplo, produtos dentro de caixas, pastas dentro de pastas, ou figuras dentro de figuras.
- A ideia central é tratar tudo como um “componente”, de modo que operações possam ser realizadas de forma recursiva sobre a estrutura completa.



Problema

- Imagine um sistema de pedidos em que há produtos e caixas.
- Cada caixa pode conter outros produtos e até outras caixas menores, que também podem conter mais itens.
- Se quisermos calcular o preço total do pedido, seria necessário percorrer manualmente cada caixa, verificar o que há dentro, somar preços e lidar com diferentes níveis de aninhamento.
- Isso torna o código complexo, frágil e difícil de manter, pois depende de muitos detalhes estruturais e da verificação constante de tipos.



Solução

- O Composite propõe criar uma interface comum, chamada Component, que define métodos como `getPrice()` ou `operation()`.
 - Os objetos simples (folhas) implementam esses métodos diretamente.
 - Os objetos compostos (contêineres), por sua vez, delegam a execução desses métodos para seus subcomponentes, somando ou combinando seus resultados.
- Essa estrutura permite percorrer toda a árvore de forma recursiva, sem precisar distinguir entre objetos simples ou compostos. Assim, o cliente pode tratar tudo como um único objeto, simplificando muito o código.

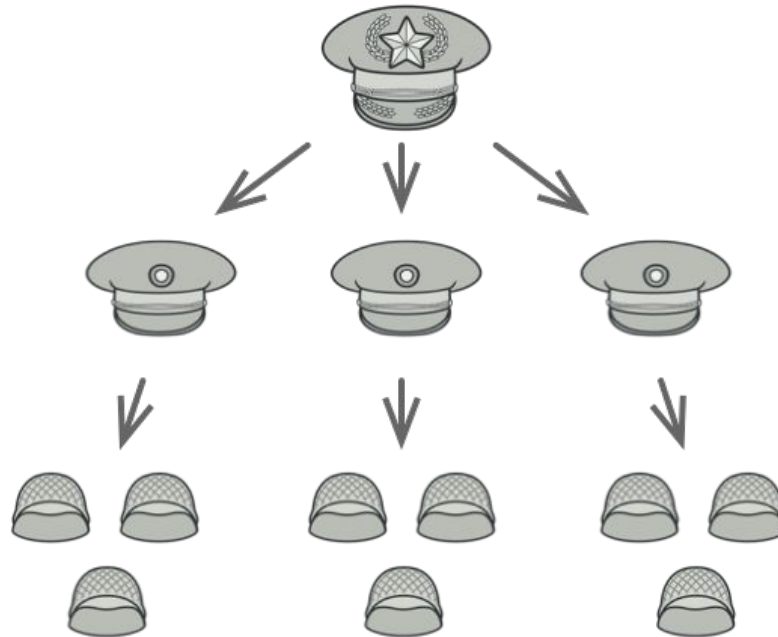


Analogia do Mundo Real

- Uma estrutura militar é uma boa analogia:
 - Um exército é dividido em divisões, que contêm brigadas, que contêm pelotões, até chegar aos soldados.
 - Uma ordem é passada do topo para baixo, e cada nível repassa a instrução aos seus subordinados.

- Da mesma forma, no padrão Composite, uma operação iniciada no topo da hierarquia (como “calcular preço” ou “desenhar”) é propagada recursivamente para cada componente.

Analogia do Mundo Real



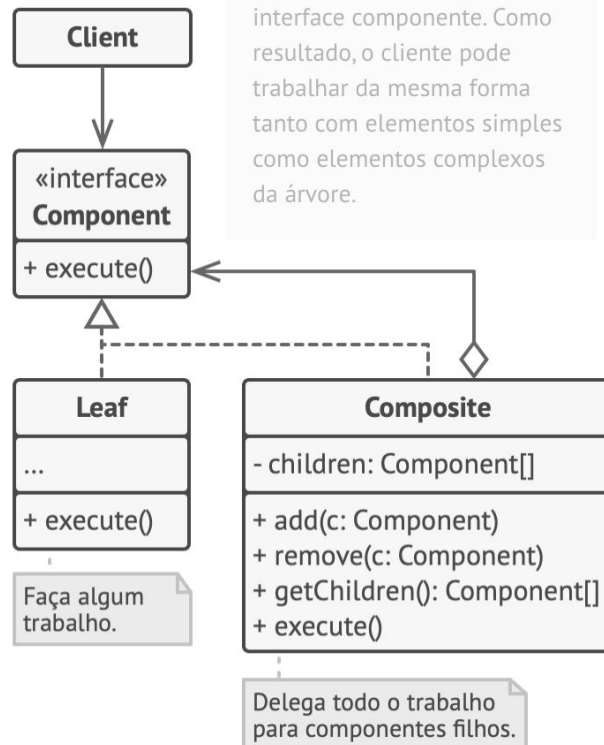
Estrutura do Padrão

- **Component** – Interface comum que define operações para todos os objetos (simples e compostos).
- **Leaf (Folha)** – Representa objetos simples, sem subcomponentes. É quem geralmente realiza o trabalho concreto.
- **Composite (Contêiner)** – Representa objetos complexos que podem conter outros componentes. Ele delega operações aos filhos e combina seus resultados.
- **Client (Cliente)** – Trabalha com todos os componentes por meio da interface Component, sem conhecer suas classes concretas.
- Essa estrutura cria uma hierarquia recursiva, onde cada Composite pode conter folhas e outros Composites.

1 A interface **Componente** descreve operações que são comuns tanto para elementos simples como para elementos complexos da árvore.

2 A **Folha** é um elemento básico de uma árvore que não tem sub-elementos.

Geralmente, componentes folha acabam fazendo boa parte do verdadeiro trabalho, uma vez que não tem mais ninguém para delegá-lo.



4 O **Cliente** trabalha com todos os elementos através da interface componente. Como resultado, o cliente pode trabalhar da mesma forma tanto com elementos simples como elementos complexos da árvore.

3 O **Contêiner** (ou *composite*) é o elemento que tem sub-elementos: folhas ou outros contêineres. Um contêiner não sabe a classe concreta de seus filhos. Ele trabalha com todos os sub-elementos apenas através da interface componente.

Ao receber um pedido, um contêiner delega o trabalho para seus sub-elementos, processa os resultados intermediários, e então retorna o resultado final para o cliente.

```
// Interface comum para todas as formas (componentes)
interface Graphic {
    fun move(x: Int, y: Int)
    fun draw()
}

// Classe folha - representa um ponto simples
class Dot(private var x: Int, private var y: Int) : Graphic {
    override fun move(x: Int, y: Int) {
        this.x += x
        this.y += y
    }

    override fun draw() {
        println("Desenhando um ponto em ($x, $y)")
    }
}

// Outra folha - representa um círculo
class Circle(x: Int, y: Int, private val radius: Int) : Dot(x, y) {
    override fun draw() {
        println("Desenhando um círculo com raio $radius")
    }
}

// Classe composta - pode conter outros objetos Graphic
class CompoundGraphic : Graphic {
    private val children = mutableListOf<Graphic>()

    fun add(child: Graphic) {
        children.add(child)
    }

    fun remove(child: Graphic) {
        children.remove(child)
    }

    override fun move(x: Int, y: Int) {
        for (child in children) {
            child.move(x, y)
        }
    }

    override fun draw() {
        println("Desenhando o grupo de formas:")
        for (child in children) {
            child.draw()
        }
        println("Fim do grupo.")
    }
}
```

Exemplo em Kotlin

- Graphic é a **interface componente**, comum a todos.
 - Dot e Circle são **folhas**.
 - CompoundGraphic é o **Composite**, que contém outros Graphics.
 - O método draw() é **recursivo** — chama draw() em cada subcomponente.
- O cliente (main) trata todos os objetos de forma uniforme.

```
from abc import ABC, abstractmethod
```

```
# Interface componente
```

```
class Graphic(ABC):
```

```
    @abstractmethod
```

```
    def move(self, x, y):
```

```
        pass
```

```
    @abstractmethod
```

```
    def draw(self):
```

```
        pass
```

```
# Classe folha - ponto
```

```
class Dot(Graphic):
```

```
    def __init__(self, x, y):
```

```
        self.x = x
```

```
        self.y = y
```

```
    def move(self, x, y):
```

```
        self.x += x
```

```
        self.y += y
```

```
    def draw(self):
```

```
        print(f"Desenhando ponto em ({self.x}, {self.y})")
```

```
# Outra folha - círculo
```

```
class Circle(Dot):
```

```
    def __init__(self, x, y, radius):
```

```
        super().__init__(x, y)
```

```
        self.radius = radius
```

```
    def draw(self):
```

```
        print(f"Desenhando círculo com centro em ({self.x}, {self.y}) e raio {self.radius}")
```

```
# Classe composta - pode conter outros objetos Graphic
```

```
class CompoundGraphic(Graphic):
```

```
    def __init__(self):
```

```
        self.children = []
```

```
    def add(self, child: Graphic):
```

```
        self.children.append(child)
```

```
    def remove(self, child: Graphic):
```

```
        self.children.remove(child)
```

```
    def move(self, x, y):
```

```
        for child in self.children:
```

```
            child.move(x, y)
```

```
    def draw(self):
```

```
        print("Desenhando grupo de formas:")
```

```
        for child in self.children:
```

```
            child.draw()
```

```
        print("Fim do grupo.\n")
```

Exemplo em Python

```
# Cliente
```

```
if __name__ == "__main__":
```

```
    dot = Dot(1, 2)
```

```
    circle = Circle(5, 3, 10)
```

```
    group = CompoundGraphic()
```

```
    group.add(dot)
```

```
    group.add(circle)
```

```
    print("=== Desenho inicial ===")
```

```
    group.draw()
```

```
    print("=== Movendo todas as formas ===")
```

```
    group.move(2, 5)
```

```
    group.draw()
```

Aplicabilidade

- Use o padrão Composite quando:
 - O modelo de dados pode ser naturalmente representado como uma árvore hierárquica (organogramas, pastas, menus, cenas gráficas, componentes visuais, etc.).
 - Você deseja que o código cliente trate objetos simples e compostos de forma uniforme, sem precisar distinguir tipos concretos.
 - Há operações que precisam ser aplicadas recursivamente sobre uma coleção de elementos aninhados.

Prós e Contras

- **Vantagens**

- Simplifica o trabalho com estruturas hierárquicas complexas.
- Usa o polimorfismo e a recursão para percorrer árvores de objetos.
- Segue o Princípio Aberto/Fechado, permitindo a adição de novos tipos de componentes sem alterar o código existente.
- Facilita a extensibilidade e reuso de código.

- **Desvantagens**

- Pode ser difícil definir uma interface comum se os componentes têm comportamentos muito distintos.
- A interface pode ficar muito genérica ou inchada, violando o princípio da segregação de interface.
- Exige cuidado na gerência de referências e composição recursiva, para evitar laços ou dependências cíclicas.



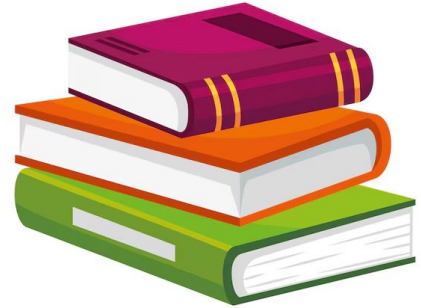
Bibliografia Básica

- MILI, Hafedh. Reuse-based software engineering: techniques, organization and measurement. New York: Wiley, 2002. 636 p. ISBN 0471398195.
- EZRAN, M.; MORISIO, M.; TULLY, C. Practical software reuse. Berlim: Springer, 2002
- POHL, K.; BÖCKLE, G.; LINDEN, F. J. Software product line engineering: foundations, principles and techniques. Berlim: Springer, c2010. xxvi, 467 p. ISBN 9783642063640 (broch.).



Bibliografia Complementar

- SOMMERVILLE, I. Engenharia de software. 7. ed. São Paulo: Pearson Addison-Wesley, 2007.
- PRESSMAN, R. Engenharia de software. 6. ed. São Paulo: Mc Graw-Hill, 2006. ISBN 8586804576
- GAMMA, E.; HELM, JOHNSON, R.; R.; VLISSIDES, J. Padrões de projeto: soluções reutilizáveis de software orientado a objetos. Porto Alegre: Bookman, 2000. 364 p.
- BRAUDE, Eric J.; BERNSTEIN, Michael E. Software engineering: modern approaches. 2nd ed. Hoboken, New Jersey: J. Wiley & Sons, 2011. xvi, 782 p.
- BASS, Len; CLEMENTS, Paul; KAZMAN, Rick. Software architecture in practice. 3rd ed. Upper Saddle River, NJ: Addison-Wesley, c2013. xix, 589 p. (SEI series in software engineering). ISBN 9780321815736 (enc.).



Obrigado!

Dúvidas?



Universidade Federal do Ceará - *Campus* Quixadá

Prof. Francisco Victor da Silva Pinheiro
victorpinheiro@ufc.br

